

UNIVERSIDAD TECNOLÓGICA DE QUERÉTARO

Unidad I · Introducción a DevOps

PROYECTO INTEGRADOR

Pipeline CI/CD automatizado para API REST

Docker · GitHub Actions · Docker Hub · AWS EC2

DATOS DE ENTREGA	INFORMACIÓN
Alumno	Axel Rodríguez
Asignatura	Introducción a DevOps
Docente	_____
Grupo	_____
Fecha	8 de octubre de 2026
Repositorio	github.com/axelisaak10/Nexa

Santiago de Querétaro, Querétaro

Índice

Núm.	Sección	Pág.
1	Introducción	3
2	Objetivo y alcance	4
3	Arquitectura de la solución	4
4	Desarrollo y calidad del backend	5
5	Pruebas automatizadas y cobertura	6
6	Contenedorización y Docker Hub	7
7	Integración continua con GitHub Actions	8
8	Despliegue continuo en AWS EC2	9
9	Resultados y análisis	10
10	Conclusiones	11
11	Bibliografía	12
12	Anexos de evidencias	13

Índice de figuras

Figura	Descripción	Pág.
1	Arquitectura del pipeline CI/CD	4
2	Pruebas automatizadas y cobertura	13
3	Colección ejecutada en Postman	14
4	Respuesta JSON del endpoint de salud	15
5	Pipeline completo en GitHub Actions	16
6	Etiquetas de la imagen en Docker Hub	17
7	Aplicación pública desplegada en AWS EC2	18
8	Instancia EC2 en ejecución	19

1. Introducción

La integración continua y el despliegue continuo constituyen prácticas centrales de DevOps porque convierten los cambios de software en versiones verificadas, reproducibles y disponibles. En un proceso manual, el equipo debe recordar cómo instalar dependencias, ejecutar pruebas, construir una imagen, etiquetarla, publicarla y sustituir la aplicación que se encuentra en producción. Cada paso manual agrega variación y aumenta el riesgo de omitir una validación, utilizar una versión incorrecta o exponer información sensible. Este proyecto transforma ese proceso en un pipeline automatizado y auditable.

La solución parte de una API REST implementada con Node.js y SQLite. El dominio utiliza categorías y productos relacionados mediante claves foráneas y ofrece doce operaciones HTTP con los métodos GET, POST, PUT y DELETE. Todas las respuestas mantienen el contrato JSON formado por statusCode y data. La API también incorpora validación de entrada, autenticación administrativa para escrituras, control de errores, respaldo de la base de datos, vaciado seguro y un endpoint de salud que actualmente responde con el mensaje hola.

La calidad se comprueba antes de construir la imagen. La suite automatizada ejecuta veinticinco pruebas unitarias y de integración para los escenarios correctos y para fallos comunes: cuerpos inválidos, identificadores inexistentes, conflictos de integridad, credenciales incorrectas y operaciones destructivas sin confirmación. La cobertura resultante supera el umbral mínimo de 70 % en líneas, ramas y funciones. También se verifican el Dockerfile, las etiquetas de la imagen y la estructura del flujo de GitHub Actions.

El pipeline se ejecuta con GitHub Actions en eventos push y pull_request dirigidos a la rama main. Primero instala las dependencias y ejecuta las pruebas con cobertura. Después de un push aprobado, construye una imagen Docker y publica dos referencias en Docker Hub: latest y el hash completo del commit. La etiqueta inmutable permite identificar exactamente qué código se desplegó, mientras que latest ofrece una referencia sencilla para la versión más reciente.

El despliegue continuo conecta el workflow con una instancia Ubuntu de Amazon EC2 mediante una llave SSH almacenada en GitHub Secrets. La instancia descarga la imagen, inicia la versión nueva, consulta /api/health y activa el tráfico público en el puerto 80 únicamente cuando la comprobación es satisfactoria. Nginx permite alternar entre posiciones azul y verde, de modo que el contenedor anterior permanece disponible durante la verificación y se reduce el tiempo de interrupción.

Este reporte documenta la arquitectura, las decisiones de implementación, las pruebas, la cobertura, la publicación en Docker Hub y el despliegue en AWS. Las evidencias se presentan como figuras numeradas y se relacionan con los resultados obtenidos. El objetivo final es demostrar que un cambio enviado al repositorio puede recorrer de forma segura todo el ciclo: validación, empaquetado, publicación y actualización automática del servidor.

2. Objetivo y alcance

Objetivo general. Implementar un pipeline CI/CD que valide una API REST, genere una imagen Docker, la publique en Docker Hub y despliegue automáticamente la versión aprobada en AWS EC2 por el puerto 80.

- Mantener al menos seis endpoints funcionales; la versión entregada contiene doce operaciones.
- Ejecutar pruebas automáticas con un umbral mínimo de cobertura de 70 %.
- Construir una imagen optimizada con Dockerfile y un contexto reducido mediante .dockerignore.
- Publicar las etiquetas latest y el hash del commit en Docker Hub.
- Desplegar en EC2 con verificación de salud y estrategia azul/verde.
- Proteger llaves, tokens, direcciones y contraseñas mediante GitHub Secrets.

3. Arquitectura de la solución

La Figura 1 resume el recorrido de un cambio desde el repositorio hasta el servicio público. Cada etapa depende del éxito de la anterior; por esta razón, una falla de pruebas impide publicar la imagen y una falla de salud impide activar la nueva versión.

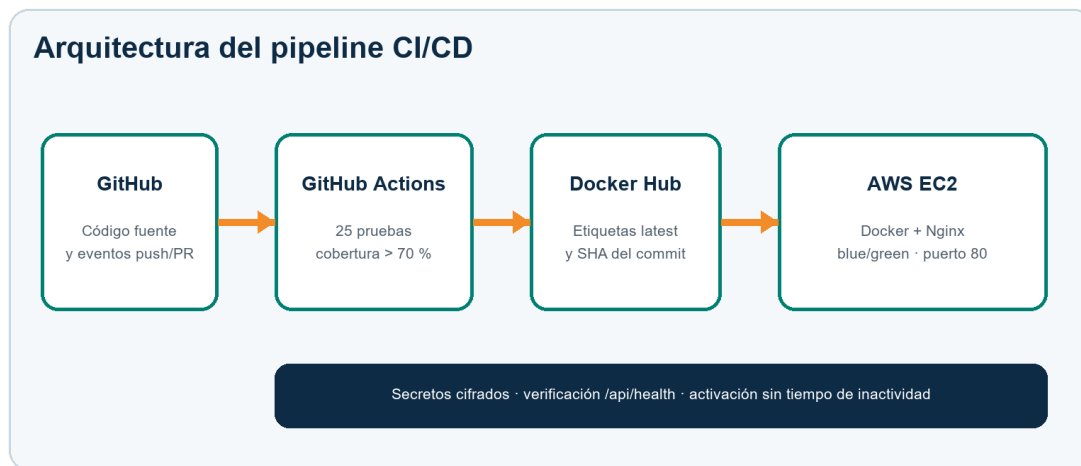


Figura 1. Arquitectura del pipeline CI/CD automatizado.

Componente	Responsabilidad	Evidencia
GitHub	Aloja el código fuente y recibe push/PR.	Repositorio Nexa
GitHub Actions	Ejecuta pruebas, build, publicación y despliegue.	Figura 5
Docker Hub	Conserva latest y la etiqueta SHA.	Figura 6
AWS EC2	Ejecuta Docker y Nginx en Ubuntu.	Figuras 7 y 8
Postman	Comprueba respuestas públicas y contratos JSON.	Figuras 3 y 4

4. Desarrollo y calidad del backend

La API administra un catálogo normalizado. Una categoría puede relacionarse con varios productos y cada producto conserva `category_id` como clave foránea. Los precios se almacenan como enteros en `price_cents` para evitar errores de redondeo. Las operaciones de escritura requieren el encabezado `X-Admin-Token` y los cuerpos se validan antes de modificar SQLite.

Método	Ruta	Operación
GET	/api/health	Estado del servicio; devuelve el mensaje hola
GET	/api/categories	Listar categorías
POST	/api/categories	Crear categoría
PUT	/api/categories/{id}	Actualizar categoría
DELETE	/api/categories/{id}	Eliminar categoría sin productos
GET	/api/products	Listar productos
POST	/api/products	Crear producto
GET	/api/products/{id}	Consultar producto
PUT	/api/products/{id}	Actualizar producto
DELETE	/api/products/{id}	Eliminar producto
POST	/api/database/backup	Crear respaldo consistente de SQLite
DELETE	/api/database	Vaciar datos y conservar la estructura

4.1 Contrato de respuesta

El contrato uniforme facilita las pruebas automáticas y el consumo desde Postman. El endpoint `GET /api/health` produce la siguiente respuesta pública, visible nuevamente en la Figura 4:

```
{
  "statusCode": 200,
  "data": [{
    "message": "hola"
  }]
}
```

4.2 Casos de error considerados

- 400: cuerpo JSON incompleto, identificador inválido o confirmación incorrecta.
- 401: ausencia del token administrativo o token inválido.
- 404: categoría o producto inexistente.
- 409: nombre duplicado o intento de eliminar una categoría con productos.
- 500: error interno controlado con la misma envoltura JSON.

5. Pruebas automatizadas y cobertura

El comando `npm run test:coverage` ejecuta la batería completa antes de que el pipeline permita construir la imagen. Las veinticinco pruebas aprobaron y no hubo casos omitidos. La Figura 2 presenta la salida del ejecutor; la Figura 3 complementa esta validación con veinte aserciones correctas realizadas sobre la colección de Postman.

Métrica	Resultado	Umbral	Evaluación
Pruebas	25 aprobadas / 0 fallidas	0 fallas	Cumplido
Líneas	93.36 %	70 %	Superado
Ramas	88.18 %	70 %	Superado
Funciones	93.55 %	70 %	Superado

Las pruebas de integración levantan el servidor con una base aislada, realizan solicitudes HTTP reales y eliminan los datos temporales al finalizar. Esto evita que el resultado dependa de la base de producción. También se verifica el socket TCP de la práctica anterior y la persistencia compartida con la API HTTP.

5.1 Escenarios representativos

- Creación, consulta, actualización y eliminación de categorías y productos.
- Rechazo de nombres duplicados y precios inválidos.
- Integridad referencial entre categorías y productos.
- Autorización de operaciones de escritura mediante `ADMIN_TOKEN`.
- Respaldo coherente y vaciado confirmado de SQLite.
- Contrato `{statusCode, data}` en respuestas exitosas y fallidas.

6. Contenedorización y Docker Hub

El Dockerfile emplea una imagen base de Node.js 22 sobre Debian slim, instala únicamente dependencias de producción, copia los archivos necesarios, crea el directorio persistente /data y ejecuta el proceso con el usuario node. El contenedor expone el puerto 80 y define una comprobación de salud sobre /api/health. El archivo .dockerignore excluye node_modules, archivos .env, registros, cobertura, reportes y datos locales.

Control	Implementación	Beneficio
Imagen base	node:22.22.3-bookworm-slim	Entorno reproducible
Usuario	node	Menor privilegio
Persistencia	/data	Conserva SQLite entre versiones
Healthcheck	GET /api/health	Detecta una versión no funcional
Contexto	.dockerignore	Reduce tamaño y exposición accidental

Después de aprobar las pruebas, el workflow publica 34axel/nexa-api:latest y 34axel/nexa-api:\${{ github.sha }}. La Figura 6 muestra ambas referencias en Docker Hub. En la ejecución documentada, el hash inicia con aba4487a y permite relacionar la imagen con el commit exacto.

```
docker pull 34axel/nexa-api:latest
docker pull 34axel/nexa-api:aba4487a19b8b37fd6def29e04b9532c8d2497ea
```

7. Integración continua con GitHub Actions

El archivo `.github/workflows/main.yml` se activa con `push` y `pull_request` hacia `main`. La ejecución registrada para el cambio `feat: devolver mensaje JSON en health` terminó con estado `Success` en 1 minuto y 6 segundos.

La Figura 5 muestra el orden y la aprobación de los tres trabajos.

Trabajo	Duración	Resultado	Acción principal
Pruebas y cobertura	17 s	Correcto	<code>npm run test:coverage</code>
Construir y publicar imagen	33 s	Correcto	Buildx y push a Docker Hub
Desplegar en AWS EC2	9 s	Correcto	SSH, healthcheck y Nginx

7.1 Gestión segura de credenciales

La configuración sensible se almacena como GitHub Secrets y no aparece en el repositorio. El workflow consume las siguientes variables únicamente durante la ejecución:

- `DOCKERHUB_USERNAME` y `DOCKERHUB_TOKEN` para publicar la imagen.
- `EC2_HOST` y `EC2_USER` para identificar el servidor de destino.
- `EC2_SSH_KEY` con el contenido privado del archivo `.pem`.
- `ADMIN_TOKEN` para proteger las rutas de escritura.
- `DEPLOY_ENABLED` como variable de control del despliegue.

La llave se escribe temporalmente con permisos restrictivos, SSH acepta la huella del servidor en el primer contacto y el archivo se elimina al terminar el trabajo.

8. Despliegue continuo en AWS EC2

La instancia Ubuntu ejecuta Docker y Nginx. El grupo de seguridad permite SSH por el puerto 22 para administración y HTTP por el puerto 80 para la API pública. Durante el despliegue se selecciona la posición inactiva, se descarga la etiqueta SHA, se inicia el contenedor nuevo y se consulta repetidamente /api/health.

Solo después de obtener HTTP 200 se genera la configuración de Nginx y se recarga el servicio. Si la comprobación falla, el contenedor nuevo se elimina y la posición anterior conserva el tráfico. La Figura 7 muestra la aplicación pública en 107.21.88.69 y la Figura 8 confirma la instancia EC2 en estado En ejecución.

Paso	Validación	Resultado esperado
1. Pull	Etiqueta SHA disponible	Imagen exacta descargada
2. Inicio	Contenedor nuevo activo	Puerto interno alterno
3. Salud	GET /api/health = 200	Respuesta {message: hola}
4. Activación	nginx -t correcto	Tráfico público al nuevo color
5. Limpieza	Versión anterior detenida	Sin contenedores obsoletos

8.1 Comprobación pública

```
curl http://107.21.88.69/api/health
{"statusCode":200,"data":[{"message":"hola"}]}
```

9. Resultados y análisis

La práctica alcanzó los resultados funcionales y de automatización solicitados. Las evidencias se concentran en las Figuras 2 a 8 y permiten seguir el recorrido desde las pruebas hasta la respuesta pública en EC2.

Requisito	Evidencia cuantitativa	Estado
API REST	12 operaciones GET, POST, PUT y DELETE	Cumplido
Pruebas automáticas	25 aprobadas; 0 fallidas	Cumplido
Cobertura	93.36 % líneas; 88.18 % ramas; 93.55 % funciones	Superado
Postman	20/20 aserciones aprobadas	Cumplido
Docker	Dockerfile y .dockerignore funcionales	Cumplido
Docker Hub	latest y SHA disponibles	Cumplido
GitHub Actions	3/3 trabajos correctos	Cumplido
AWS EC2	HTTP 200 en puerto 80	Cumplido
Health	JSON con message: hola	Cumplido
Seguridad	Secretos fuera del código público	Cumplido

9.1 Análisis

El umbral de cobertura no solo se alcanzó; se superó por más de dieciocho puntos en la métrica más baja. Esto ofrece margen para detectar regresiones, aunque no sustituye la revisión del comportamiento real. Por esa razón, la colección de Postman y la verificación pública complementan la suite interna.

El uso conjunto de latest y SHA resuelve dos necesidades distintas: facilidad de uso y trazabilidad. EC2 despliega la referencia SHA, por lo que una investigación posterior puede identificar el commit exacto. La estrategia azul/verde agrega una barrera de protección: el tráfico cambia únicamente después de validar la nueva versión.

La ejecución exitosa de GitHub Actions prueba la integración de las cuatro plataformas. El cambio del mensaje de salud llegó al repositorio, superó las pruebas, produjo una imagen nueva, fue publicado y finalmente respondió como JSON desde AWS. Este resultado satisface la demostración solicitada de realizar un git push y observar la actualización automática.

10. Conclusiones

El proyecto integró desarrollo, pruebas, contenedorización y nube en un flujo único. La API no depende de una preparación manual para demostrar su calidad: cada cambio en main ejecuta veinticinco pruebas y aplica umbrales de cobertura antes de producir un artefacto. La cobertura obtenida, superior a 88 % en todas las métricas, confirma que se evaluaron tanto los caminos exitosos como errores frecuentes del consumidor.

Docker Hub funciona como punto de intercambio entre integración y despliegue. Las etiquetas latest y SHA permiten disponer de la versión más reciente sin perder trazabilidad. El Dockerfile reduce privilegios, conserva SQLite en un volumen persistente y proporciona una comprobación de salud. Estas decisiones hacen que el mismo artefacto aprobado en GitHub Actions sea el que se ejecuta en EC2.

El despliegue azul/verde mejora la disponibilidad porque separa la preparación de la activación. El nuevo contenedor recibe tráfico únicamente después de responder HTTP 200, mientras Nginx conserva la posibilidad de mantener la versión anterior ante una falla. La evidencia pública del mensaje hola demuestra que el cambio atravesó el pipeline completo y no quedó limitado al entorno local.

La gestión de secretos completa la solución desde el punto de vista operativo. Los tokens de Docker Hub, la llave .pem, el host y el token administrativo permanecen fuera del código. Como mejora futura, conviene agregar HTTPS con un dominio, monitoreo de métricas y alertas, rotación periódica de credenciales y una base administrada cuando el volumen de información exceda las necesidades de SQLite.

11. Bibliografía

Las fuentes se consultaron el 8 de octubre de 2026. Se priorizó documentación oficial para mantener precisión técnica.

1. Amazon Web Services. (2026). Get started with Amazon EC2. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/EC2_GetStarted.html
2. Amazon Web Services. (2026). Security group rules. <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/security-group-rules.html>
3. Docker. (2026). Dockerfile reference. <https://docs.docker.com/reference/dockerfile/>
4. Docker. (2026). Build and push Docker images with GitHub Actions. <https://docs.docker.com/build/ci/github-actions/>
5. Docker. (2026). Personal access tokens. <https://docs.docker.com/security/access-tokens/>
6. GitHub. (2026). Understanding GitHub Actions. <https://docs.github.com/actions/about-github-actions/understanding-github-actions>
7. GitHub. (2026). Using secrets in GitHub Actions. <https://docs.github.com/actions/security-guides/using-secrets-in-github-actions>
8. Node.js. (2026). Test runner and collecting code coverage. <https://nodejs.org/api/test.html>
9. Nginx. (2026). Beginner's guide: configuration and reload. https://nginx.org/en/docs/beginners_guide.html
10. SQLite. (2026). SQLite backup API. <https://www.sqlite.org/backup.html>

12. Anexos de evidencias

Las ocho figuras siguientes fueron mencionadas y analizadas en las secciones anteriores. En conjunto documentan las pruebas, el registro de imágenes, la automatización y la infraestructura en la nube.

Anexo A. Pruebas automatizadas

```
● ● ● Proyecto Integrador CI/CD - Pruebas y cobertura

$ npm run test:coverage
[+] Configuración CI/CD del proyecto integrador
  [+] workflow push y pull_request a main
  [+] Docker Hub con secrets y tags latest + SHA
  [+] despliegue SSH blue-green con health check
[+] Integración de 12 endpoints, errores y persistencia
  [+] 01 GET health y doce operaciones documentadas
  [+] GET y POST de categorías y productos
  [+] 07 PUT categories actualiza datos
  [+] 08 PUT products actualiza datos
  [+] Rechazo de duplicados, FK inválida y eliminación del padre
  [+] Esquemas, autorización y errores de usuario
  [+] Respaldo, persistencia, DELETE y vaciado seguro
  [+] Socket TCP 6061: insert y get comparten SQLite
[+] tests 25      pass 25      fail 0      skipped 0
[+] lines 93.29% | branches 88.18% | functions 93.55%
```

Figura 2. Veinticinco pruebas aprobadas y cobertura superior al 70 %.

Anexo B. Pruebas en Postman

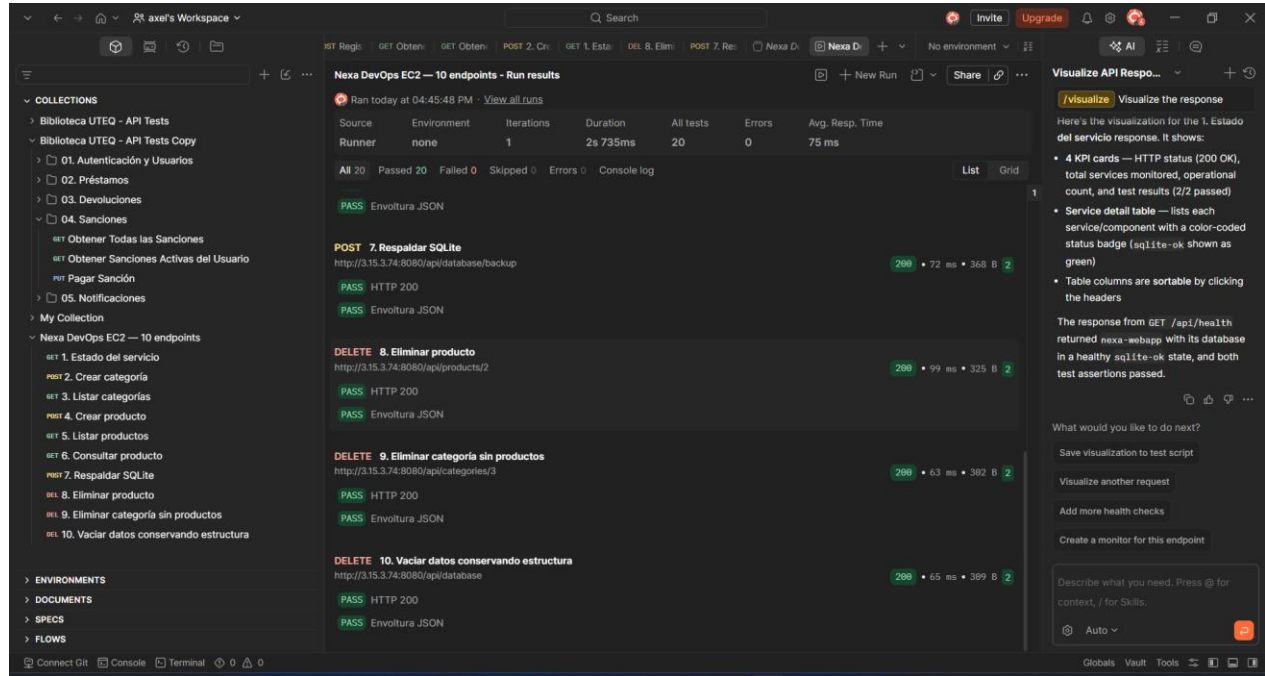


Figura 3. Ejecución de la colección con veinte aserciones aprobadas y cero errores.

La ejecución recorre los endpoints en orden y valida el código HTTP y la envoltura JSON. El resultado confirma que la API puede consumirse desde una herramienta externa.

Anexo C. Respuesta pública en Postman

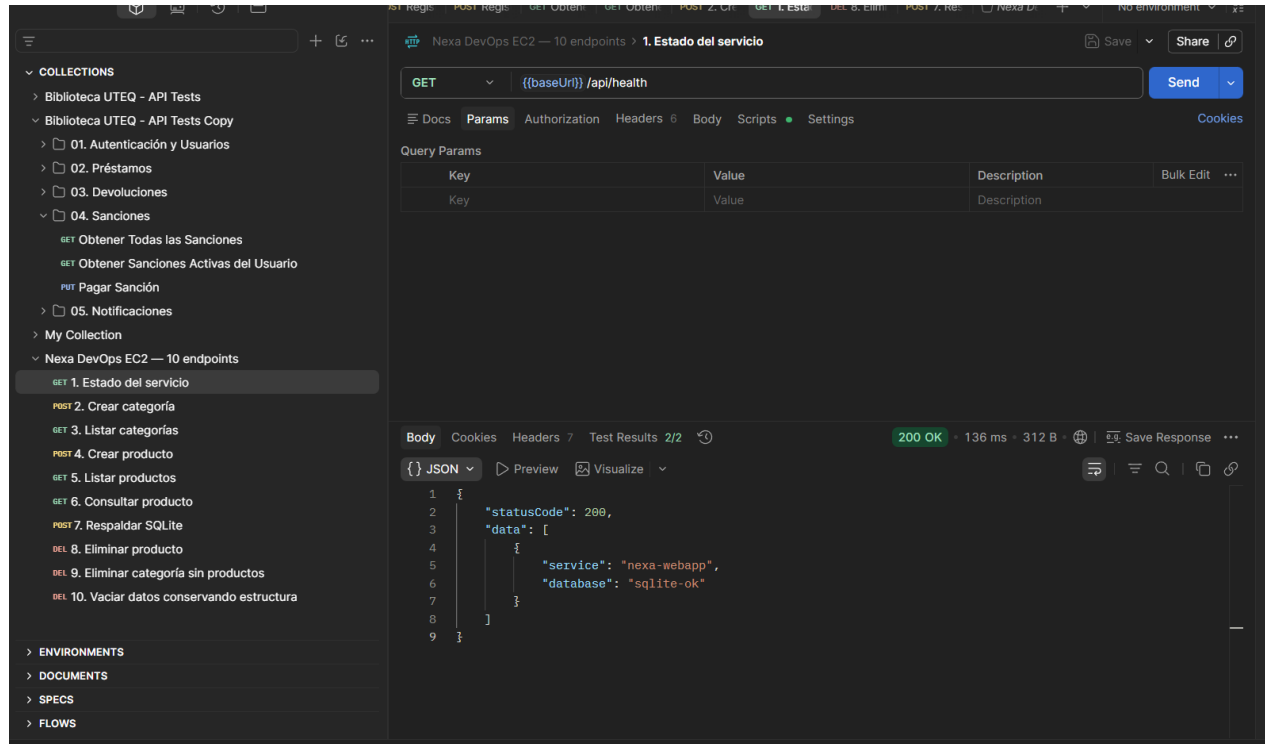


Figura 4. Respuesta HTTP 200 del endpoint de salud consultado desde Postman.

Después de la actualización, el mismo endpoint conserva el contrato {statusCode, data} y entrega el mensaje hola. La respuesta exacta también se documenta en las secciones 4.1 y 8.1.

Anexo D. Ejecución del pipeline

axelisaak10 / Nexa

<> Code Issues Pull requests **Actions** Projects Security and quality Insights

← API CI CD

✔ feat: devolver mensaje JSON en health #9 Sign in to view logs

Summary

All jobs

- ✔ Pruebas y cobertura
- ✔ Construir y publicar imagen
- ✔ Desplegar en AWS EC2

Run details

- Usage
- Workflow file

Triggered via push 14 hours ago

axelisaak10 pushed · aba4487 main

Status: Success Total duration: 1m 6s Artifacts: 1

main.yml
on: push

```

graph LR
    A[✔ Pruebas y cobertura 17s] --> B[✔ Construir y publicar imagen 33s]
    B --> C[✔ Desplegar en AWS EC2 9s]
  
```

Annotations
3 warnings and 3 notices

⚠ Pruebas y cobertura
Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, actions/setup-node@v4

Figura 5. Tres trabajos correctos en GitHub Actions: pruebas, publicación y despliegue.

El resumen muestra el estado Success, el commit aba4487a y el flujo secuencial completado en 1 minuto y 6 segundos.

Anexo E. Registro de contenedores

Explore / 34axel / nexa-api

34axel/nexa-api
By 34axel · Updated about 15 hours ago

IMAGE

☆ 0 ↓ 567

Overview **Tags**

Sort by **Newest** Filter tags

Digest	OS/ARCH	Compressed size
aba4487a19b8b37fd6def29e04b9532c8d2497ea	linux/amd64	76.19 MB
latest		

By clicking "Accept All Cookies", you agree to the storing of cookies on your device to enhance site navigation, analyze site usage, and assist in our marketing efforts.

[Cookies Settings](#) [Reject All](#) [Accept All Cookies](#)

Figura 6. Docker Hub con las etiquetas del hash del commit y latest.

La etiqueta SHA distingue cada versión y evita depender únicamente de una referencia mutable.

Anexo F. Aplicación desplegada

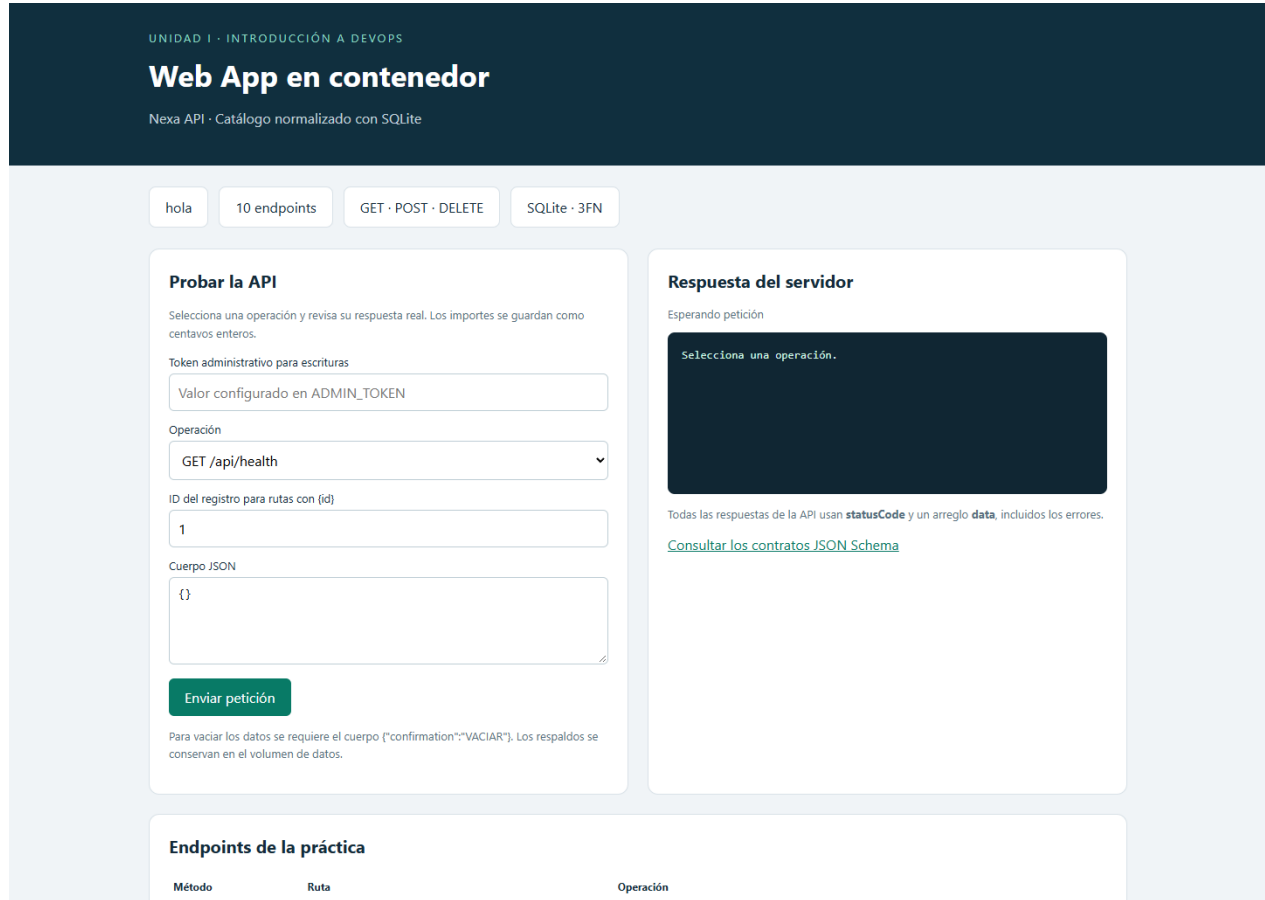


Figura 7. Interfaz pública servida desde AWS EC2 por el puerto 80.

La tarjeta superior muestra hola y la tabla documenta las doce operaciones disponibles en la API.

Anexo G. Infraestructura AWS

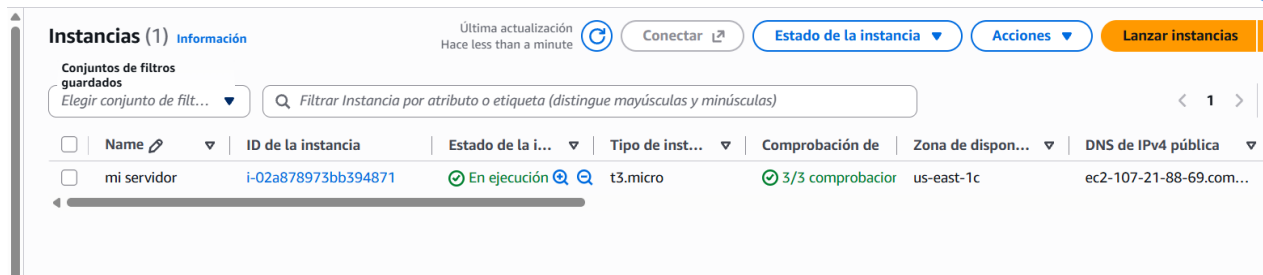


Figura 8. Instancia EC2 Ubuntu en ejecución y con comprobaciones aprobadas.

La instancia aloja Docker y Nginx. El acceso público se concentra en HTTP 80 y la administración utiliza SSH 22.